

GharKhana — Project Details

The Idea

GharKhana was born from a simple observation — every city in India has hundreds of incredible home cooks who make food that no restaurant can replicate, yet they have no platform to reach customers. At the same time, thousands of people crave authentic home-cooked meals but have no way to find them.

GharKhana bridges that gap. It is a neighbourhood-scale, pre-order based home food delivery platform that connects verified home cooks with customers who want real, fresh, homemade food — delivered to their door in a scheduled time slot.

This is not Swiggy. This is not Zomato. This is something different.

The Problem

For customers:

- Restaurant food is expensive, processed, and repetitive
- Tiffin services are rigid — same food, same time, no choice
- There is no platform to discover and order from home cooks nearby
- "Ghar ka khana" craving has no digital solution

For home cooks:

- Talented cooks (mostly women, mostly 40–65 years old) have no platform to monetise their skill
 - Word of mouth is their only marketing channel
 - No system for taking orders, managing payments, or organising delivery
 - They cook for 5 families when they could cook for 25
-

The Solution

GharKhana is a **pre-order, slot-based home food delivery platform** with four distinct user roles and a complete operational backbone.

How it works:

1. Home cooks register, get approved by admin, and set up their kitchen profile
2. They publish daily menus (Lunch/Dinner) the night before with a cutoff time
3. Customers browse nearby cooks, view menus, and pre-order by the cutoff time

4. The cook prepares fresh food the next morning
 5. A dedicated delivery person collects and delivers in optimized batches
 6. Customers receive food in their chosen time slot with PIN confirmation
-

Who It's For

Primary users:

- Customers aged 22–45 in urban/semi-urban Indian neighbourhoods who crave home food
- Home cooks aged 35–65 (housewives, retired individuals) who want to monetise their cooking

Secondary users:

- Delivery persons assigned to individual kitchens
 - Platform admin managing operations
-

Core Features

Customer Side

- GPS-based cook discovery using Haversine formula
- Location pinning with Leaflet + OpenStreetMap (no Google Maps API cost)
- Browse menus filtered by Veg/Non-Veg/Egg, Lunch/Dinner, distance
- Pre-order system with cutoff time enforcement
- Same-day ordering (cook-controlled, capacity-limited)
- Cart with live fee calculation (subtotal + 0.2% platform fee + tiered delivery charge)
- Two payment modes — Online (UPI/card) and Cash on Delivery
- 4-digit PIN system for secure delivery confirmation
- Order status tracking — Confirmed → Preparing → Out for delivery → Delivered
- Post-delivery rating and written review system
- Multiple saved addresses with default selection

Cook Side

- Admin-approved onboarding with kitchen GPS pinning
- Permanent dish library — create once, reuse forever
- Daily menu builder with Draft → Publish workflow

- Capacity management with hard daily limits
- Same-day order toggle with 35% capacity cap
- Configurable order cutoff times
- Cook dashboard — live orders, earnings, capacity bar
- Separate pre-order and same-day order management
- Full order status control (Confirm → Preparing → Out for delivery)

Delivery Side

- Route optimization using Nearest Neighbor algorithm (pure Python, no API)
- Haversine formula for accurate distance calculation between GPS points
- Delivery batches grouped by time slot
- Google Maps multi-stop URL generated automatically (no API key)
- One-at-a-time delivery card flow
- Per-stop Google Maps navigation link
- PIN confirmation for online-paid orders
- Cash collected confirmation for COD orders
- Failed delivery reporting with photo upload
- Photo notification sent to prepaid customers on failed delivery

Admin Side

- Cook approval queue with profile review
- Platform-wide order monitoring with filters
- Revenue analytics — platform fees, commission tracking
- Monthly commission invoice generation (automated + manual trigger)
- Mark invoices as paid with payment reference logging

Billing System

- 0.2% platform fee charged to customers at checkout
- 5% monthly commission on cook's gross earnings (online + COD)
- Automated invoice generation via Django management command
- UPI Pay button integration for cook commission payments
- Cook bank detail storage (UPI ID, account number, IFSC)
- 15-day payment window with due date tracking

Notification System

- In-app notification system for all order status changes
- Cook notified on new orders, reviews
- Customer notified on order confirmation, preparation, dispatch, delivery
- Failed delivery photo notification to prepaid customers
- Invoice generation notifications to cooks

Technical Architecture

Stack

Layer	Technology
Backend	Django 4.2 (Python)
Database	MySQL 8.0+
Frontend	HTML5, CSS3, Vanilla JavaScript
Templates	Django Template Engine
Maps (customer)	Leaflet.js + OpenStreetMap (free, no API key)
Maps (delivery)	Google Maps URL API (free, no key needed)
Authentication	Custom Django User model with RBAC
Location	Haversine formula — pure Python implementation
Route optimization	Nearest Neighbor algorithm — pure Python
Payments	UPI redirect (live), Razorpay (integration-ready)
File storage	Django media files (local, upgradeable to S3)

Database — 15 MySQL tables

Table	Purpose
users	Custom user model with 4 roles
cook_profiles	Kitchen details, GPS, capacity, settings
saved_addresses	Customer delivery addresses with GPS

Table	Purpose
delivery_persons	Linked to cook, handles deliveries
cook_bank_details	UPI ID and bank details
dishes	Cook's permanent dish library
delivery_slots	Configurable delivery time windows
daily_menus	Published daily menus (Lunch/Dinner)
menu_items	Dishes within a menu with quantity
orders	Complete order record with all calculations
order_items	Snapshot of dishes ordered (price + name frozen at order time)
deliveries	Delivery records with sequence and PIN
failed_deliveries	Failed delivery records with photo
reviews	Customer reviews (unique per order)
commission_invoices	Monthly billing records
notifications	In-app notification records

Key Algorithms

Haversine Formula (distance calculation)

Used for:

- Cook discovery (sort by distance from customer)
- Delivery charge calculation (₹15 / ₹30 / ₹50 tiers)
- Self-pickup eligibility (within 1km)
- Route optimization (distance between stops)

Implementation: Pure Python, no external library

Accuracy: $\pm 0.5\%$ for neighbourhood-scale distances

Nearest Neighbor Algorithm (route optimization)

Used for: Delivery route optimization within each time slot

Process:

1. Start at cook's kitchen GPS coordinates
2. Calculate Haversine distance to all unvisited stops
3. Visit the nearest unvisited stop
4. From that stop, find next nearest
5. Repeat until all stops visited

Output:

- Ordered delivery sequence
- Per-leg distance (from previous stop)
- Total route distance
- Estimated delivery time (distance ÷ 20 km/h + 5 min per stop)
- Google Maps multi-stop URL

Complexity: $O(n^2)$ – optimal for $n \leq 30$ stops (neighbourhood scale)

Business Model

Revenue streams

1. Platform fee (0.2%) Charged to customer at checkout on every order Transparent line item in bill summary Example: ₹200 order → ₹0.40 platform fee

2. Monthly commission (5%) Charged to cook on total gross earnings per month Calculated on 1st of every month automatically Includes both online and COD orders Example: Cook earns ₹40,000/month → ₹2,000 commission

Unit economics example

Cook capacity:	20 meals/day
Average order:	₹180
Daily revenue:	₹3,600
Monthly revenue:	₹1,08,000
Platform commission (5%):	₹5,400/month per cook
Platform fee (0.2%):	₹216/month per cook
Total per cook:	₹5,616/month

At 50 active cooks: ₹2,80,800/month platform revenue

Business Rules (Enforced in Code)

Rule	Implementation
Pre-orders close at cook's cutoff time	Server-side time check in order placement view
Same-day orders need 2-hour buffer	Datetime comparison against slot start time
Daily capacity cannot be exceeded	Atomic count check before order creation
Same-day orders $\leq$ 35% of capacity	<code>floor(daily_capacity × 0.35)</code> checked separately
Self-pickup only within 1km	Haversine check at cart stage
Delivery charge by distance tier	Calculated server-side, never trusting client
Platform fee calculated server-side	<code>subtotal × 0.002</code> , never from client input
One review per order	<code>unique_together(order, customer)</code> at DB level
Menu draft not visible to customers	Status filter on all customer-facing menu queries
Order item prices frozen at order time	<code>dish_name</code> and <code>unit_price</code> snapshotted on OrderItem
Default address — only one per customer	Overridden in <code>SavedAddress.save()</code> method
Pre-orders auto-confirm	Status set to CONFIRMED immediately on order creation
Same-day orders manual	Status set to PENDING, cook must confirm

Design System

Fonts: Syne (display, headings) + DM Sans (body, UI) **Primary:** Saffron orange #E8590C
Background: Warm off-white #F5F0EB **Card surface:** #FFFDFB with 1.5px warm border
Food type indicators: Green (Veg), Red (Non-veg), Amber (Egg) — Indian standard **Design language:** Modern & bold, warm, neighbourhood-scale — not a corporate startup

What Makes This Different

Feature	GharKhana	Swiggy/Zomato	Tiffin services
Food source	Home cooks	Restaurants	Central kitchen
Order model	Pre-order (next day)	Instant	Subscription

Feature	GharKhana	Swiggy/Zomato	Tiffin services
Delivery	Slot-based batch	30 minutes	Fixed daily
Personalization	Cook profile, menu choice	Menu browsing	Fixed menu
Cook empowerment	Full platform	No	No
Price	₹80-200/meal	₹150-400/meal	₹100-150/day
Authenticity	Real home food	Restaurant food	Industrial
Route optimization	Nearest Neighbor	Advanced ML	Manual

Current Status

Component	Status
User authentication (4 roles)	✔ Complete
Cook onboarding + kitchen setup	✔ Complete
Dish library management	✔ Complete
Daily menu builder	✔ Complete
Customer cook discovery	✔ Complete
Order placement + checkout	✔ Complete
Order tracking	✔ Complete
Cook dashboard	✔ Complete
Delivery route optimization	✔ Complete
PIN confirmation system	✔ Complete
Failed delivery workflow	✔ Complete
Review and rating system	✔ Complete
Billing and commission system	✔ Complete
Admin panel	✔ Complete
In-app notifications	✔ Complete

Component	Status
Landing page	✓ Complete
Online payment (UPI redirect)	✓ Live
Razorpay integration	🚧 Integration-ready
Email notifications	📄 Planned
Mobile app	📄 Planned
Cook analytics dashboard	📄 Planned

Roadmap

Phase 1 — Launch (Now)

- Onboard 5-10 pilot cooks in one neighbourhood
- Acquire first 50 customers through WhatsApp and local outreach
- Run operations manually alongside the platform to catch edge cases

Phase 2 — Stabilise (Month 2-3)

- Razorpay full integration
- Email/SMS notifications via Twilio
- Cook earnings analytics with charts
- Customer reorder with one click
- Mobile-responsive refinements

Phase 3 — Scale (Month 4-6)

- Expand to 3 neighbourhoods
- WhatsApp Business API for order updates
- Cook leaderboard and badges
- Bulk ordering for offices
- Progressive Web App (PWA)

Phase 4 — Growth (Month 7-12)

- Multi-city expansion
- Native mobile apps (Android first)
- Subscription meal plans

- Cook training and quality programme
 - Investor ready metrics dashboard
-

The Team Needed to Scale

Currently built by: 1 developer (full-stack) **To scale, needs:**

- 1 operations manager (cook onboarding, quality)
- 1 marketing person (WhatsApp, Instagram, local outreach)
- 1 delivery coordinator (per city)

Technology team to add later:

- 1 backend engineer (payments, scale)
 - 1 mobile developer (Android app)
-

The Vision

GharKhana is not trying to be the next Zomato.

GharKhana is trying to be the platform that gives Savitri Devi — 58 years old, cooking for 35 years, feeding her family with love — a way to feed 25 more families and earn a dignified income doing what she already does every single day.

That's the mission. Every line of code serves that mission.

Ghar ke haath ka jadoo — digitised.